WEBRUM-04: Session Analysis

> **Series:** WEBRUM | **Notebook:** 4 of 8 | **Created:** March 2026 | **Last Updated:** 03/12/2026

Overview

User session analysis is the foundation of understanding how real users interact with your web applications. A session represents a complete visit — from the first page load to the last interaction before timeout. By analyzing sessions, you can identify user journey patterns, measure engagement, track conversions, calculate bounce rates, and understand how geographic location and device type impact the user experience.

This notebook covers session segmentation, user journey mapping, conversion funnel analysis, bounce rate calculations, and geographic/device breakdowns using DQL.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
RUM Enabled	Web applications with session capture active
Session Properties	Custom session properties configured (optional but recommended)
Permissions	`storage:events:read`
Previous Notebook	WEBRUM-01: RUM Fundamentals

1. Session Properties

Dynatrace automatically captures standard session properties and allows you to define custom ones:

Standard Properties

Property	Description	Example Values
`session.id`	Unique session identifier	`abc123def456`
`user.type`	Session classification	`REAL_USER`, `ROBOT`, `SYNTHETIC`
`app.name`	Application name	`MyWebApp`
`duration`	Total session duration	`300000000000` (nanoseconds)
`action.count`	Number of user actions	`15`
`error.count`	Number of errors in session	`2`
`country`	User's country	`United States`
`city`	User's city	`Chicago`
`os.family`	Operating system	`Windows`, `macOS`, `iOS`
`browser.family`	Browser	`Chrome`, `Firefox`, `Safari`
`screen.width`	Screen width in pixels	`1920`
`screen.height`	Screen height in pixels	`1080`
`connection.type`	Network connection type	`4g`, `wifi`, `ethernet`

Custom Session Properties

Define custom properties via **Settings > Web and mobile monitoring > Session and user action properties**. Common examples:

- `session.user_id` — Authenticated user identifier
- `session.plan_type` — Subscription tier (free, pro, enterprise)
- `session.cart_value` — Shopping cart total
- `session.ab_variant` — A/B test variant

```
// Explore session data — view available fields
fetch dt.rum.user_session, from:-1h
| filter user.type == "REAL_USER"
| limit 5
```

2. Session Segmentation

Segmenting sessions helps identify patterns across different user groups. Common segmentation dimensions include engagement level, error impact, and return frequency.

```
// Engagement segmentation — bucket sessions by number of actions
fetch dt.rum.user_session, from:-24h
| filter user.type == "REAL_USER"
| fieldsAdd engagement = if(action.count == 1, "Bounce",
    else: if(action.count &lt;= 3, "Low (2-3 actions)",
    else: if(action.count &lt;= 10, "Medium (4-10 actions)",
    else: "High (10+ actions)")))
| summarize session_count = count(),
    avg_duration = avg(duration),
    avg_errors = avg(error.count),
    by:{engagement}
| sort session_count desc
```

```
// Error-impacted sessions — how many sessions had errors?
fetch dt.rum.user_session, from:-24h
| filter user.type == "REAL_USER"
| summarize total_sessions = count(),
    error_sessions = countIf(error.count &gt; 0),
    by:{app.name}
| fieldsAdd error_session_pct = round(toDouble(error_sessions) /
toDouble(total_sessions) * 100.0, decimals: 1)
| sort error_session_pct desc
```

3. User Journey Mapping

Understanding the sequence of actions within sessions reveals common navigation paths, dead ends, and drop-off points.

```
// Top entry pages — where do users start their journey?
fetch user.events, from:-24h
| filter action.type == "Load"
| summarize first_action = takeFirst(action.name), by:{session.id}
| summarize entry_count = count(), by:{first_action}
| sort entry_count desc
| limit 10
```

```
// Top exit pages — where do users leave?
fetch user.events, from:-24h
| filter action.type == "Load"
| summarize last_action = takeLast(action.name), by:{session.id}
| summarize exit_count = count(), by:{last_action}
| sort exit_count desc
| limit 10
```

```
// Session duration distribution by hour of day — when are users most active?
fetch dt.rum.user_session, from:-7d
| filter user.type == "REAL_USER"
| fieldsAdd hour = getHour(timestamp)
| summarize session_count = count(),
    avg_duration_sec = avg(toDouble(duration) / 1000000000.0),
    by:{hour}
| sort hour asc
```

4. Conversion Tracking

Conversion tracking identifies which sessions completed a desired business action (purchase, signup, form submission). This requires either:

- **Session properties** marking conversion events
- **Specific user actions** that indicate conversion (e.g., a "Thank You" page load)

Conversion via Action Name Detection

If your conversion page has a recognizable URL pattern, you can detect conversions from user actions:

```
// Conversion rate — sessions that reached a checkout/confirmation page
// Adapt the action.name filter to match your conversion page
fetch dt.rum.user_session, from:-24h
| filter user.type == "REAL_USER"
| summarize total_sessions = count(),
    by:{app.name}
| append [
    fetch user.events, from:-24h
    | filter action.type == "Load"
    | filter action.name ~ "*confirmation*" or action.name ~ "*thank*you*" or
action.name ~ "*checkout*success*"
    | summarize converted_sessions = countDistinct(session.id), by:{app.name}
  ]
```

> **Tip:** For accurate conversion tracking, use Dynatrace session properties to flag conversion events. This avoids reliance on URL pattern matching, which can be fragile.

5. Bounce Rate Analysis

A "bounce" is a session with only one user action — the user loaded one page and left. High bounce rates may indicate poor landing page relevance, slow performance, or broken functionality.

```
// Bounce rate by application
fetch dt.rum.user_session, from:-24h
| filter user.type == "REAL_USER"
| summarize total_sessions = count(),
    bounced_sessions = countIf(action.count == 1),
    by:{app.name}
| fieldsAdd bounce_rate_pct = round(toDouble(bounced_sessions) /
toDouble(total_sessions) * 100.0, decimals: 1)
| sort bounce_rate_pct desc
```

```
// Bounce rate trend over 7 days — track improvement over time
fetch dt.rum.user_session, from:-7d
| filter user.type == "REAL_USER"
| fieldsAdd is_bounce = if(action.count == 1, 1, else: 0)
| makeTimeseries total = count(), bounces = sum(is_bounce), interval:1d
| fieldsAdd bounce_rate = arrayAvg(bounces) / arrayAvg(total) * 100.0
```

6. Geographic Analysis

RUM data includes geographic information derived from the user's IP address. This helps identify regional performance differences and target optimization efforts.

```
// Sessions by country — top 15 countries by volume
fetch dt.rum.user_session, from:-24h
| filter user.type == "REAL_USER"
| filter isNotNull(country)
| summarize session_count = count(),
    avg_actions = avg(action.count),
    avg_errors = avg(error.count),
    avg_duration_sec = avg(toDouble(duration) / 1000000000.0),
    by:{country}
| sort session_count desc
| limit 15
```

```
// Top 10 cities by session count with average performance
fetch dt.rum.user_session, from:-24h
| filter user.type == "REAL_USER"
| filter isNotNull(city) and isNotNull(country)
| summarize session_count = count(), by:{city, country}
| sort session_count desc
| limit 10
```

7. Device and Browser Analysis

Understanding the device and browser mix helps prioritize testing and optimization efforts.

```
// Session distribution by browser
fetch dt.rum.user_session, from:-24h
| filter user.type == "REAL_USER"
| filter isNotNull(browser.family)
| summarize session_count = count(),
    avg_errors = avg(error.count),
    by:{browser.family}
| sort session_count desc
| limit 10
```

```
// Session distribution by operating system
fetch dt.rum.user_session, from:-24h
| filter user.type == "REAL_USER"
| filter isNotNull(os.family)
| summarize session_count = count(),
    avg_duration_sec = avg(toDouble(duration) / 1000000000.0),
    by:{os.family}
| sort session_count desc
```

```
// Screen resolution distribution — identify common viewport sizes
fetch dt.rum.user_session, from:-24h
| filter user.type == "REAL_USER"
| filter isNotNull(screen.width) and isNotNull(screen.height)
| fieldsAdd resolution = concat(toString(screen.width), "x",
toString(screen.height))
| summarize session_count = count(), by:{resolution}
| sort session_count desc
| limit 10
```

8. Summary and Next Steps

In this notebook, we covered:

- **Session properties** — Standard and custom properties for segmentation
- **Engagement segmentation** — Bucketing sessions by interaction depth
- **User journey mapping** — Entry/exit pages and activity by time of day
- **Conversion tracking** — Identifying sessions that completed business goals
- **Bounce rate analysis** — Measuring and trending single-action sessions
- **Geographic analysis** — Session distribution by country and city
- **Device/browser analysis** — Understanding the technology mix of your users

Next Steps

- **WEBRUM-05: Error Analysis** — JavaScript error tracking and impact analysis
- **WEBRUM-06: Performance Analysis** — Deep dive into page load waterfall timings

References

- Session Properties

- User Session Query

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.